箕面ハイキングマップ 公開API 仕様書（契約バージョン 3.1）

バージョン: 2.1 **最終更新日:** 2026年9月24日 **対象:** minoh-hiking `api/` — 実装側 / MapPublisher — 呼び出し側 **関連:** 機能仕様 `funcspec-202611.md` §10（契約 1.0 の §10.2 は**本書で置き換えた**） / MapPublisher `docs/migration-plan-202608.md`（移行の実装計画）

---

1. 本書について

1.1 目的

本書は、ハイキングマップアプリが表示するデータを外部の運用アプリから公開し、利用者アプリへ配信するための **公開API の契約**と、**配信される GeoJSON の形式（公開スキーマ）**を定義する。

1.2 正本

本書が正本である。 API の実装（`api/`）と同じリポジトリに置く。

- 呼び出し側（MapPublisher）は本書に依存し、**検証ルールを再実装しない**
- 検証は API に任せ、失敗時は API が返した日本語メッセージをそのまま表示する
- 契約と公開スキーマを1冊にまとめるのは、**検証ルールとスキーマがずれることを防ぐ**ため

破壊的変更のときだけ契約バージョンを上げ、呼び出し側にも反映する。

1.3 契約 3.0 からの変更点（3.1）

追加のみで、既存の呼び出しは変わらない。 `reopenDate` を送らない公開は 3.0 と同じく受理される。

項目	内容
公開スキーマ	closures の Feature に **`reopenDate`（解除予定日・任意）を追加**（§3.3）。通行止め・通行困難のどちらにも付けられる
検証	`reopenDate` があるときは **`YYYY-MM-DD` 形式の実在する日付**であること（§6.3）。違反は 400

利用者アプリは、値があるときだけポップアップに「解除予定」を表示する。 3.1 に対応していない版のアプリは `reopenDate` を無視するだけなので、データを先に公開しても表示は崩れない。

1.4 契約 2.1 からの変更点（3.0）

破壊的変更。 `version` の設定者がサーバーから送信側へ移る。

項目	契約 2.1	契約 3.0	理由
`version` の設定者	サーバーが採番（送られても無視）	**送信側が body に入れて送る（必須）**	連番を意図的に飛ばす運用（`.08` の次を `.10` にするなど）を可能にする

項目	契約 2.1	契約 3.0	理由
形式	`mapdata`/`tiles` は `yyyy.n`、`closures` は `yyyy-mm.n`	**全データセットで `yyyy.nn`**（2桁ゼロ埋め・月区切りは廃止）	様式を1つに揃える。ゼロ埋めにより文字列の大小比較が版の順序と一致する
重複した version	起こり得ない（採番のため）	**400 で拒否する**（§6.5）	同じ version では利用者アプリが更新に気づけない（§10）

`version` を送らない、または形式が違うリクエストは 400 になる。 **契約 2.1 までの呼び出し側は、そのままでは公開できない。**

> **なぜ重複拒否が必須なのか。** 利用者アプリの更新判定は**等値比較のみ**である（§10）。 同じ version のまま内容だけ変えて公開すると、公開は成功したように見えるのに、 利用者の端末は更新に気づかない。採番をやめる以上、この事故はサーバーで止めるほかない。

1.5 契約 2.0 からの変更点（2.1）

追加のみで、既存の呼び出しは変わらない。 mapdata / closures の送信・応答・検証は 2.0 のまま。

項目	内容
データセット	**`tiles`（オフライン地図のタイル一覧）を追加**。GeoJSON ではない
公開スキーマ	§3.6 にタイル一覧の構造を追加。`FeatureCollection` の要件は GeoJSON データセットにのみ適用する
検証	§6.1 を「GeoJSON データセット共通」に改め、§6.4 に tiles 固有のルールを追加
エンドポイント	`GET/POST /api/tiles` を追加

タイル一覧はこれまでアプリに同梱していた（`public/data/tile_manifest.json`）。 範囲を変えるたびにアプリのデプロイが必要だったため、他の2つと同じ配信に揃える。

1.6 契約 1.0 からの変更点

項目	契約 1.0	契約 2.0	理由
データセット	`closures` のみ	**`mapdata` を追加**（緊急ポイント・ルート・スポット）	アプリ同梱の geojson を公開経由の更新に変更する
受け付ける geometry	`Point` のみ	**`Point` と `LineString`**	ルートが `LineString` のため
`version`	クライアントが body に入れて送る（必須）	**サーバーが採番する**（送られても無視する）	上げ忘れによる「公開したのに届かない」を防ぐ

項目	契約 1.0	契約 2.0	理由
更新の通知	なし（毎回フル取得）	`GET /api/manifest` で version のみ先読み	mapdata が約 200KB あり、毎起動のフル取得を避ける
履歴	`history/` に無制限に追加	**前回分1世代のみ**（上書き）	保存量が公開回数に比例して増え続けるため
公開トークン	`CLOSURES_PUBLISH_TOKEN`	`MAP_PUBLISH_TOKEN`（2データセット共通）	対象が closures だけではなくなるため

2. データセット

2.1 一覧

データセット	日本語名	内容	version 形式	想定更新頻度
`mapdata`	**ハイキングマップデータ**	緊急ポイント・ハイキングルート・スポット	`yyyy.nn`	年に数回
`closures`	**通行止め・通行困難地点**	通行止め・通行困難の地点	`yyyy.nn`	随時
`tiles`	**オフライン地図のタイル一覧**	ダウンロード対象の地理院タイル（z/x/y）	`yyyy.nn`	年に1回程度

`mapdata` / `closures` は GeoJSON、**`tiles` は GeoJSON ではない**（§3.6）。

日本語名は利用者向けの画面・文書で用いる呼称であり、**本書を正本とする**。

2.2 責務分担

主体	責務
MapEditor	地点・ルート・スポットの編集、通行止め地点の登録。作業用 GeoJSON を出力する
DownloadArea	オフライン地図の対象範囲を指定し、`tile_manifest.json` を出力する。**公開はしない**
MapPublisher	作業用 GeoJSON を**公開スキーマへ整形**し、公開API へ送信する
公開API（本書）	検証・version 採番・保存・配信
minoh-hiking アプリ	配信データの表示。**GET のみ**利用する

編集用の識別子（`spot` の `id` など）は MapEditor の作業ファイルには残し、 **MapPublisher が公開時に落とす**。編集の都合と配信の都合を分離する。

3. 公開スキーマ

配信されるデータの形式を定義する。**利用者の表示に必要なものだけを出す。**

§3.1〜§3.5 は GeoJSON データセット（`mapdata` / `closures`）に適用する。 `tiles` は GeoJSON ではないため §3.6 に別途定義する。

3.1 FeatureCollection（GeoJSON データセット）

```
{
  "type": "FeatureCollection",
  "version": "2026.01",
  "updatedAt": "2026-08-18T05:12:34.567Z",
  "features": [ ... ]
}
```

キー	型	必須	設定者
`type`	`"FeatureCollection"`	○	送信側
`version`	string（`yyyy.nn`）	○	**送信側**（§4）
`updatedAt`	string（ISO 8601）	○	**サーバー**（公開時刻）
`features`	array	○	送信側

`version` は**送信側が必ず入れる**（契約 3.0）。形式が違う、または現在公開中と同じ値なら 400 になる（§4.3）。 `updatedAt` は送信側が入れても**サーバーの値で上書きされる**。

3.2 mapdata の Feature

3.2.1 緊急ポイント（`ポイントGPS`）

プロパティ	型	必須	説明
`type`	`"ポイントGPS"`	○	固定値
`id`	string	○	例 `"B-01"`。**現地の標識と対応する利用者向けの識別子**のため保持する
`name`	string	○	例 `"聖天展望台"`
`description`	string	—	**値があるときのみ出力する**（null は出力しない）

geometry: `Point`

> `pointId` は**出力しない**。全件で `id` と同値のため。

3.2.2 スポット（`spot`）

プロパティ	型	必須	説明
`type`	`"spot"`	○	固定値
`name`	string	○	例 `"滝道32鉄橋"`

geometry: `Point`

> `id` / `source` / `description` は**出力しない**。
>
> - `id` はどこからも参照されていない（ルートはスポットを `name` で参照する）。表示に必要なのは `name` と座標のみ
> - `source` は全件が `"image_transformed"`、`description` は全件が `"スポット（GPS変換済）"` の定数

スポット名は一意ではない（「トイレ」「WC」など）。名称の重複は許容し、識別は座標で行う。

3.2.3 ルート（`route`）

プロパティ	型	必須	説明
`type`	`"route"`	○	固定値
`id`	string	○	例 `"route_H-04_to_H-11"`。区間の識別に使う
`startPointGPS`	`[経度, 緯度, 標高?]` | null	○	開始点の座標
`endPointGPS`	`[経度, 緯度, 標高?]` | null	○	終了点の座標

geometry: `LineString`（**中間点のみ**。開始点・終了点の座標は含まない）

> `startPoint` / `endPoint`（ID参照）は**出力しない**。 端点の座標が入っているため ID による解決は不要であり、区間の識別は `id` で足りる。

利用者アプリは `startPointGPS` を `LineString` の先頭に、`endPointGPS` を末尾に補って描画する。 `null` の場合は補わない（その端は中間点から始まる）。

3.3 closures の Feature（`closure`）

プロパティ	型	必須	説明
`type`	`"closure"`	○	固定値
`id`	string	○	全地点で一意
`name`	string	○	地点名
`kind`	`"closed"` | `"difficult"`	○	通行止め / 通行困難
`reason`	string	—	工事・倒木・落石 など。値があるときのみ
`note`	string	—	備考。値があるときのみ

プロパティ	型	必須	説明
relatedRoute	string	—	値があるときのみ
reopenDate	string（YYYY-MM-DD）	—	解除予定日。通行止め・通行困難のどちらにも付けられる。値があるときのみ（契約 3.1〜）
updatedAt	string	○	地点ごとの更新日時

geometry: Point

3.4 座標

- 形式: [経度, 緯度] または [経度, 緯度, 標高]
- 測地系: WGS84
- **小数点以下5桁**に丸める（約1m精度）
- 標高の単位: メートル

3.5 出力例（mapdata）

```
{
  "type": "FeatureCollection",
  "version": "2026.01",
  "updatedAt": "2026-08-18T05:12:34.567Z",
  "features": [
    {
      "type": "Feature",
      "properties": { "type": "ポイントGPS", "id": "B-01", "name": "聖天展望台" },
      "geometry": { "type": "Point", "coordinates": [135.47258, 34.839, 184.6] }
    },
    {
      "type": "Feature",
      "properties": { "type": "spot", "name": "滝道32鉄橋" },
      "geometry": { "type": "Point", "coordinates": [135.47102, 34.84955, 162.3] }
    },
    {
      "type": "Feature",
      "properties": {
        "type": "route",
        "id": "route_H-04_to_H-11",
        "startPointGPS": [135.4713, 34.87451, 534],
        "endPointGPS": [135.47341, 34.86829, 552.9]
      },
      "geometry": {
        "type": "LineString",
        "coordinates": [[135.47142, 34.87301], [135.47205, 34.87088]]
      }
    }
  ]
}
```

3.6 tiles の構造（GeoJSON ではない）

DownloadArea が出力した `tile_manifest.json` をそのまま送る。整形は不要。

```
{
  "version": "2026.01",
  "updatedAt": "2026-08-21T04:00:00.000Z",
  "source": "download-area-edited",
  "layers": {
    "z14_default":  { "z": 14, "buffer_m_max": 300, "tile_count": 8,   "tiles":
[[14357, 6497], ...] },
    "z18_optional": { "z": 18, "buffer_m_max": 150, "tile_count": 824, "tiles":
[[229724, 103959], ...] }
  }
}
```

プロパティ	型	必須	説明
`layers`	object	○	レイヤーキー → レイヤー定義。1つ以上
`layers.<key>.z`	number	○	ズームレベル（10〜18）
`layers.<key>.tiles`	array	○	`[x, y]` の配列
`layers.<key>.tile_count`	number	—	あれば `tiles` の要素数と一致すること
`source`	string	—	出力元の記録。無ければ空文字で保存する

- `version` / `updatedAt` は**送られても無視し、サーバーの値を採用する**（他のデータセットと同じ）
- レイヤーキー（`z14_default` 等）は利用側アプリが「基本／詳細」の区分に使う。 **サーバーはキーの命名を検証しない**

---

4. version

4.1 形式

全データセットで `yyyy.nn`。 `nn` は**2桁ゼロ埋め**の連番とする。

例	意味
`2026.01`	2026年の1回目
`2026.10`	2026年の10回目
`2027.01`	年が変わって1回目

- `nn` が3桁になる形式（`2026.100`）は**受け付けない**。年に100回の公開は想定しない。
- 契約 2.1 まであった `closures` の月区切り（`yyyy-mm.n`）は**廃止**した。 公開月は `updatedAt` から分かるため、番号に持たせない。

4.2 決めるのは送信側

サーバーは採番しない。 送信側（MapPublisher）が `version` を body に入れて送る（§3.1・§3.6）。

送信側は「現在公開中の version から `nn` を1加算した値」を既定値として画面に表示し、 **運用者が手で変更できる**ようにする。連番を意図的に飛ばす運用（`.08` の次を `.10` にする）を 可能にするためである。既定値の算出規則は呼び出し側に置く（本書は形式と受け入れ条件のみを定める）。

4.3 サーバーが見るのは形式と重複だけ

条件	応答
`yyyy.nn` 形式でない、または `version` が無い	400 `version は yyyy.nn 形式で指定してください(例: 2026.01)`
`manifest.json` の現在値と**同一**	400 `version {値} はすでに公開されています。番号を進めてください`
上記以外	受け付ける

巻き戻し（小さい番号での公開）は拒否しない。 意図的な差し戻しを止めないためである。 誤った巻き戻しの検知は、呼び出し側の確認ダイアログ（件数差分）が担う。

4.4 比較

`nn` をゼロ埋めするため、**文字列の大小比較がそのまま版の順序と一致する** （`2026.09 < 2026.10`）。契約2.1 以前の `yyyy.n` にあった `2026.10 < 2026.9` の逆転は起きない。

ただし**更新判定は等値比較（`!==`）で行う**（§10）。順序を見る必要がなく、 差し戻しでも確実に更新が届くためである。

---

5. エンドポイント

5.1 一覧

メソッド	パス	認証	用途
GET	`/api/manifest`	不要	全データセットの version・件数（数百バイト）
GET	`/api/mapdata`	不要	mapdata の GeoJSON
POST	`/api/mapdata`	要	mapdata の公開（全置換）
GET	`/api/closures`	不要	closures の GeoJSON
POST	`/api/closures`	要	closures の公開（全置換）
GET	`/api/tiles`	不要	tiles のタイル一覧（JSON）
POST	`/api/tiles`	要	tiles の公開（全置換）
OPTIONS	全て	不要	CORS プリフライト（204）

上記以外のメソッドは `405 Method Not Allowed`。

5.2 GET /api/manifest

```
{
  "mapdata":  { "version": "2026.03", "updatedAt": "2026-08-18T05:12:34.567Z",
"count": 668 },
  "closures": { "version": "2026.01", "updatedAt": "2026-08-17T22:03:11.004Z",
"count": 12  },
  "tiles":    { "version": "2026.01", "updatedAt": "2026-08-21T04:00:00.000Z",
"count": 1248 }
}
```

`count` は GeoJSON データセットでは Feature 数、**tiles では全レイヤーのタイル枚数の合計**。

- `Cache-Control: no-store`
- `manifest.json` が未作成・取得失敗のときは、各データセットの本体から復元して返す
- それも取得できないデータセットは `{ "version": "", "updatedAt": null, "count": 0 }` を返す

5.3 GET /api/mapdata, GET /api/closures, GET /api/tiles

- `Content-Type`: GeoJSON は `application/geo+json; charset=utf-8` / tiles は `application/json; charset=utf-8`
- `Cache-Control: no-store`（キャッシュは端末側が担う）
- Blob 未作成・取得失敗時も **200 で空を返す**。アプリの表示を止めないため
  - GeoJSON: `{"type":"FeatureCollection","version":"","features":[]}`
  - tiles: `{"version":"","updatedAt":null,"source":"","layers":{}}`

5.4 POST /api/mapdata, POST /api/closures, POST /api/tiles

- リクエスト: `Content-Type: application/json`、ヘッダ `x-publish-token`
- ボディ: 公開スキーマ（§3）
  - `mapdata` / `closures`: FeatureCollection（§3.1）
  - `tiles`: タイル一覧（§3.6）
- **保存は全置換**（0件を送ると全消去になる）
- **`version` は必須**（`yyyy.nn`）。形式違い・現在公開中と同一なら 400（§4.3）
- `updatedAt` は送られても無視し、サーバーの値を採用する
- `count` は GeoJSON では Feature 数、tiles では全レイヤーのタイル枚数の合計

成功応答（200）:

```
{ "ok": true, "dataset": "mapdata", "version": "2026.03", "count": 669,
  "updatedAt": "2026-08-18T05:12:34.567Z" }
```

5.5 エラー応答

ステータス	意味	`error` の内容	呼び出し側の扱い
400	検証エラー	日本語の説明	データを直して再実行
401	トークン不正	固定文言	保存済みトークンを消して再入力
405	未対応メソッド	固定文言	—
500	保存失敗	日本語の説明	時間をおいて再実行（§7.2 のとおり冪等）
503	サーバー側トークン未設定	固定文言	操作では直らない。設定が必要

呼び出し側は `error` の文言を**そのまま表示する**。文言を独自に持たない。

---

6. 検証ルール

問題がなければ受理し、あればエラーメッセージを返す。**判定はここにのみ置く。**

`version` の検証（形式・重複）は全データセット共通で、データの検証より**先に**行う（§4.3・§7.2）。

6.1 GeoJSON データセット共通（`mapdata` / `closures`）

1. `FeatureCollection` 形式であり `features` が配列であること
2. 各要素が `Feature` 形式で `geometry` を持つこと
3. 座標が箕面エリアの範囲内であること
   - 経度 `135.2`〜`135.8` / 緯度 `34.6`〜`35.1`
   - **`LineString` は全頂点を検査する**
   - `startPointGPS` / `endPointGPS` も同様に検査する（`null` は許容）
4. `id` が一意であること
   - **`id` が未設定の Feature はスキップする。** `spot`（§3.2.2）は `id` を持たないため

6.2 mapdata 固有

- `properties.type` が `ポイントGPS` / `route` / `spot` のいずれかであること
- geometry は `Point` または `LineString` であること
  - `ポイントGPS` / `spot` は `Point`
  - `route` は `LineString`

6.3 closures 固有

- geometry は `Point` のみ
- `reopenDate` があるときは、`YYYY-MM-DD` 形式で暦の上で実在する日付であること（`2026-02-30` や `2026/10/31` は不可。`null` は未設定として扱う）

6.4 tiles 固有

GeoJSON ではないため §6.1 は適用しない。

1. `layers` がオブジェクトで、1つ以上のレイヤーを持つこと
2. 各レイヤーの `z` が整数で **10〜18** の範囲であること（アプリの minZoom / maxZoom）
3. 各レイヤーの `tiles` が配列であること

4. `tile_count` があれば `tiles` の要素数と一致すること
    - **途中で切れたファイルに気づくための照合。** 一致しなければ 400
5. 各タイルが `[x, y]` の整数で、`0 <= x, y < 2^z` であること
6. 各タイルが**箕面エリアに掛かること**
    - タイルが覆う経緯度の範囲が §6.1-3 と同じ枠（経度 `135.2`〜`135.8` / 緯度 `34.6`〜`35.1`）と 重なるかで判定する。掛からないタイルは入力ミスとして 400

`id` の一意性は対象外（タイル一覧に `id` は無い）。

---

7. 保存仕様

7.1 Blob 構成

```
manifest.json                            ← 採番の基準・GET /api/manifest の実体
mapdata/minoh-hiking-mapdata.geojson     ← 現行
mapdata/previous.geojson                 ← 前回分（1世代のみ・毎回上書き）
closures/minoh-hiking-closure.geojson    ← 現行
closures/previous.geojson                ← 前回分（1世代のみ・毎回上書き）
tiles/tile_manifest.json                 ← 現行
tiles/previous.json                      ← 前回分（1世代のみ・毎回上書き）
```

7.2 POST の書き込み順

1. `manifest.json` を読み、`version` の形式と重複を検証する（§4.3）
2. データを検証する（§6）
3. **現行の本体を `previous` へ退避する**
4. 本体を put する
5. `manifest.json` を put する
6. 200 を返す

手順5で失敗した場合は 500 を返す。 `manifest.json` が進んでいないため、**同じ version でもう一度公開できる**（重複判定も通る）。運用者は「もう一度公開」で復旧できる（冪等）。

手順3で失敗しても**公開は成立させる**（警告ログのみ）。退避の失敗で公開を止めるほうが、運用上の害が大きい。手順3が成功して手順4が失敗した場合、`previous` が現行と 同一内容になるだけで害はない。

7.3 前回分の保持

```
// @vercel/blob の copy()。本体をダウンロードせずに退避できる
await copy(BLOB_PATH, PREVIOUS_PATH, {
  access: 'public',
  contentType: 'application/geo+json',  // copy は metadata を引き継がないため再指定する
  addRandomSuffix: false,
  allowOverwrite: true
});
```

初回公開時は本体が存在せず `BlobNotFoundError` になる。**握りつぶして続行する。**

---

8. 認証

- 環境変数 `MAP_PUBLISH_TOKEN`（Vercel に設定。コミット禁止）
- ヘッダ `x-publish-token`
- 比較は**固定長ハッシュ同士**で行う（SHA-256 → `timingSafeEqual`）。タイミング攻撃対策
- 環境変数が未設定のときは 503 を返す
- 2つのデータセットで**同一のトークン**を使う

---

9. CORS

- `Access-Control-Allow-Origin: *`
- `Access-Control-Allow-Methods: GET, POST, OPTIONS`
- `Access-Control-Allow-Headers: Content-Type, x-publish-token`

GET は公開データであり、POST はトークンで保護されるため Origin を限定しない。 GitHub Pages 版アプリ・MapPublisher からもクロスオリジンで参照するため許可が必要。 **呼び出し側に CORS 対応の追加実装は不要。**

---

10. 利用側アプリの更新判定

利用者アプリ（minoh-hiking）は **GET のみ**を使う。

```
1. localStorage から保存済み version を読む
2. Cache API から GeoJSON を読んで即描画          ← オフラインでもここまでで表示される
3. GET /api/manifest（cache: 'no-store'）
     ├ 失敗（オフライン等）        → 終了
     ├ version が一致              → 終了（本体を取りに行かない）
     └ version が相違
          4. GET 本体
          5. Cache API に保存
          6. 再描画
          7. localStorage の version を更新    ← ★必ず最後
```

手順7を先に行ってはならない。 本体の取得・保存に失敗したあとに version だけが進むと、 以後その端末は永久に更新されなくなる。

一度もオンラインで起動していない端末は表示なしとなる。 その状態では地図タイルも未取得のため、運用上の問題としない。

---

11. 実装しないこと

意図的に持たない。**将来「不足している」と見えても、以下の理由で追加しない。**

項目	理由
誤公開に対する件数下限チェック	公開前の確認は呼び出し側のダイアログで行う。サーバー側に閾値判定を持つと、正当な一括削除ができなくなる
呼び出し側での検証の追加	判定は本書の §6 にのみ置く。二重管理は必ずずれる
本書の内容の呼び出し側への転記	同上。呼び出し側は**依存している項目だけ**を列挙し、ルールは書き写さない
公開前の実機プレビュー	「公開 → アプリで確認 → 必要なら再公開」の運用とする。全置換＋前回分の保持があるため復旧できる
データセットごとのトークン分離	運用者は1名。共通トークンで足りる

12. 変更履歴

版	日付	内容
1.0	2026-08-18	初版。契約バージョン 2.0 を定義し、機能仕様書 §10.2（契約 1.0）を置き換える
2.1	2026-09-24	**契約バージョン 3.1**。closures の Feature に `reopenDate`（解除予定日・`YYYY-MM-DD`・任意）を追加し（§3.3）、形式の検証を追加（§6.3）。追加のみで既存の呼び出しに影響なし（§1.3）
2.0	2026-08-23	**契約バージョン 3.0（破壊的変更）**。`version` の設定者をサーバーから**送信側**へ移し、形式を全データセットで **`yyyy.nn`**（2桁ゼロ埋め）に統一。`closures` の月区切り（`yyyy-mm.n`）を廃止。現在公開中と同一の `version` を **400 で拒否**する規則を追加（§4.3）。§4 を「採番」から「version」へ全面改訂し、§3.1・§5.4・§6・§7.2 を追随させた
1.1	2026-08-21	**契約バージョン 2.1**。データセット `tiles`（オフライン地図のタイル一覧）を追加（追加のみ・既存の呼び出しに影響なし）。GeoJSON 以外を扱うため §3.6 に構造、§6.4 に検証ルールを追加し、§6.1 を「GeoJSON データセット共通」に改めた。責務分担に DownloadArea を追加（出力のみ・公開は MapPublisher）